June 2026

OpenWorld on ARC-AGI-3: Synthesizing Verified Code World Models for Interactive Reasoning

James Schwoebel[1,*] • Ingrida Semenec, PhD[1] • Jenia Rousseva, PhD[1] • Marcos Ortiz, PhD[1] • Collin Overbay[1] • Manish Bhatt[4,6] • Rome Thorstenson[3] • Martin G. Frasch, MD, PhD[2,5]

[1] Quome, Inc. [2] University of Washington [3] Rafter Labs, Inc. [4] OWASP [5] Health Stream Analytics, LLC
[6] IEEE Computer Society [*] Corresponding author

ABSTRACT

ARC-AGI-3 is the ARC Prize Foundation's interactive-reasoning benchmark: an agent is dropped into an unfamiliar grid game, must discover the rules *by acting*, and reach goals over many steps—testing exploration, *modeling*, goal-setting, and planning. Frontier agents score near zero. We argue this is precisely the regime where a *world computer* should win, and give the OpenWorld recipe: **explore** a game to gather verified (s, a, s') transitions, **synthesize** the game's dynamics as *verified code* (an LLM writes a `predict(frame, action)` function, accepted only after it exact-matches held-out transitions), and **plan** through the verified model to reach goals. The wager is the same one that powers symbolic code world models elsewhere: a *correct* code model is exact and admits search, where a learned next-frame model compounds error. A precondition holds empirically—on the public game `ls20`, observed dynamics are *deterministic* (every repeated state–action pair yields an identical next frame across hundreds of transitions), so the dynamics are writable as exact code; states are dense (∼1,500 of 4,096 cells non-background) but per-step changes are sparse (∼50 cells), which a code model handles natively while a token-level learner cannot. Across the 25 public games, the precondition is *universal* (determinism 1.00 on every game), and synthesizing with a *frontier* model recovers verified dynamics: on the 21 real-dynamics games the synthesized code reaches 49% mean held-out exact-match, beating the copy-frame baseline on 19/21 and a local code model (qwen-32B, 12%; Claude wins 16/17), with 3 *near-perfect* (≥90%) world models. **Synthesizer capability is decisive.** Solving (level completion) remains the open frontier—0 levels, as for frontier agents—so the contribution is that ARC-AGI-3 dynamics are recoverable as *verified code*, not that the games are solved.

Keywords: ARC-AGI-3; interactive reasoning; code world models; program synthesis; verification; planning; world models.

1 Introduction

The interactive frontier. ARC-AGI-1 and -2 measure abstraction from static input–output grids; ARC-AGI-3 moves to *interactive* environments where the rules and goals are revealed only through action. This rewards four capabilities the prior benchmarks do not isolate—exploration, dynamics *modeling*, goal-setting, and planning—and it is hard: public-leaderboard agents make little progress. Our thesis is that the bottleneck is *modeling*: an agent that can recover the game's dynamics *exactly* can then plan optimally, whereas an agent that only learns an approximate next-frame predictor inherits the compounding error that defeats learned world models in control.

This paper. We port OpenWorld's verified-code-world-model recipe to ARC-AGI-3. The contribution is (i) the *recipe*—explore, synthesize-and-verify code dynamics, plan—and the argument for why it fits an interactive grid benchmark; (ii) the empirical *precondition* that the games are deterministic and therefore code-expressible; and (iii) [in progress] a measured comparison of level completion against random and learned-model baselines on the public games.

2 The ARC-AGI-3 setting

Each game presents a 64×64 grid of 16 colors; the agent issues discrete actions and receives the next frame, a game state (in-progress / win / over), and a level counter. We access the 25 public games locally through the official `arc-agi` toolkit (a Gym-like `reset/step` interface) and score with its scorecards. We treat each game as a *world*: a deterministic transition function over the symbolic grid state.

3 The recipe: explore, synthesize-and-verify, plan

Explore. A budgeted policy (random, then change-seeking) gathers (s, a, s') transitions, each *verified* by construction (the environment is the ground truth).

Synthesize verified code. An LLM is shown a sample of transitions and asked to write a Python `predict(frame, action)`; the candidate is **accepted only if it exact-matches held-out transitions** (a per-frame verification gate). Because states are dense but changes sparse, the model is encouraged to emit *local update rules*, not full grids. Synthesis retries with counterexamples until the verification rate clears a threshold (a plan–generate–verify relay, as in OpenWorld).

Plan. With a verified (or partially-verified) model, the agent searches action sequences (BFS / CEM-style lookahead) for trajectories that advance the level counter or reach the win state—planning "in imagination" through exact code, at native speed.

4 Results: verified-code fidelity (E86)

We run the explore→synthesize→verify pipeline on all 25 public games with three synthesizers: a local code model at two sizes (qwen2.5-coder 7B, 32B) and a frontier model (Claude). Fidelity is *held-out transition exact-match*: the fraction of unseen (frame, action) pairs whose *entire* next 64×64 frame the synthesized `predict` reproduces exactly. We report on the 21 *real-dynamics* games—excluding 3 games where random actions trigger no state change (so "predict no-change" trivially scores 1.0) and 1 game (`tn36`) where random play never produced a valid step.

The precondition is universal. Replay-determinism is 1.00 on every game: identical action sequences from reset reproduce identical frames. ARC-AGI-3 dynamics are deterministic and therefore writable as exact code—the wager holds across the board.

A frontier synthesizer recovers verified dynamics. Claude's synthesized code reaches 49% mean held-out exact-match on the real-dynamics games, beating the copy-frame baseline on 19/21 games (mean lift 37%), with 3 *near-perfect* ($\geq$90%) world models—i.e. code that essentially *is* the

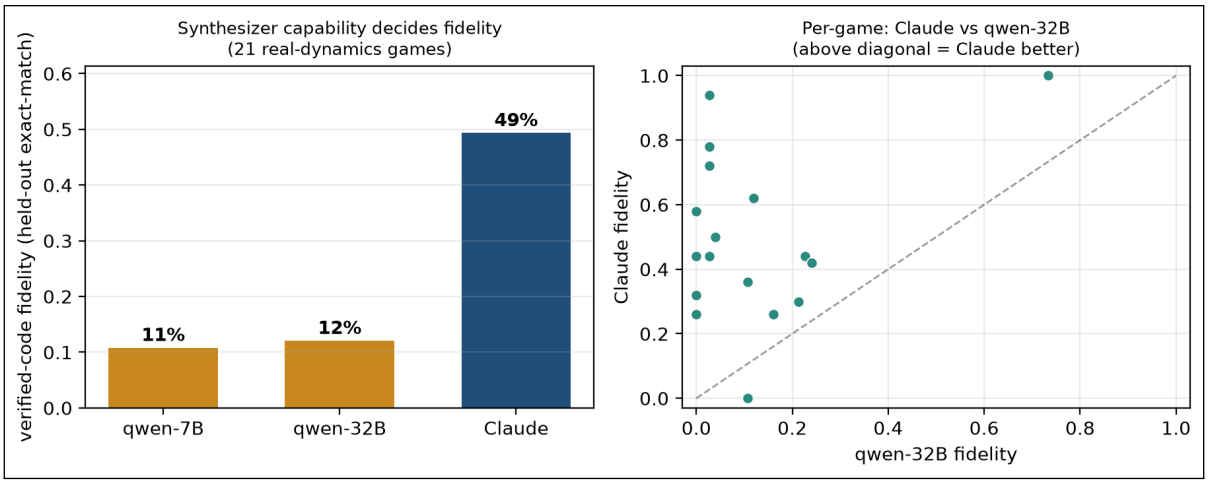

Figure 1: **E86 verified-code fidelity on ARC-AGI-3 (21 real-dynamics games).** *Left:* mean held-out exact-match by synthesizer—qwen-7B ≈ qwen-32B ≪ Claude. *Right:* per-game Claude vs. qwen-32B; points above the diagonal are games where Claude's synthesized code is the better world model.

game's transition function. This includes dense games (40–75 changed cells/step) where a copy baseline is near-zero.

Synthesizer capability is decisive. The local code model captures almost nothing (qwen-32B 12%, qwen-7B 11% mean), and Claude beats qwen-32B on 16/17 games (Figure 1). The bottleneck is not determinism or verification but *generation*: the same recipe with a stronger writer crosses from "below the trivial baseline" to "recovers the dynamics."

Honest limits. (i) **Solving is unsolved**: 0 levels completed by any arm—fidelity is not yet a solution, and ARC-AGI-3 remains hard (frontier agents also score near zero). (ii) The densest game (`cn04`, ∼174 changed cells/step) defeats even Claude (fidelity 0). (iii) On one game (`tr87`) qwen edged Claude. (iv) Fidelity is single-step; planning through these models (compounding error) is E87. These bound the claim to what we measured: *ARC-AGI-3 dynamics are recoverable as verified code by a capable synthesizer*—a foundation for, not yet a demonstration of, solving.

5 Agentic synthesis and the capability ceiling (E86b)

Does a better *harness* close the synthesizer gap? We replace one-shot prompting with a verifier-in-the-loop (generate → run the code → read the exact failing cells → revise), run *identically* for qwen-30B and Claude across 15 games (Table 1). The agentic loop lifts Claude (49% → 53%, helping 7/15 games—e.g. pushing `s5i5` to a *perfect* model) but does **nothing** for qwen-30B (8% → 7%). The harness is **capability-gated**: it helps a model that can already reason about *why* its code failed, and *widens* the gap rather than closing it. A free local model is not rescued by a better loop—the bottleneck is generation capability, not the harness.

Table 1: **The 2×2 (verified-code fidelity, 15 games).** Synthesizer capability dominates (∼7×); the agentic loop is capability-gated—it helps Claude, not qwen.

synthesizer	one-shot	agentic loop
qwen-30B	8%	7%
Claude	49%	53%

6 Planning through the model (E87)

A verified model is only useful if you can *search* it. We test **controllability**: sample a reachable target (a short random action sequence from reset), plan a sequence *through the verified code* to reach it, then execute in the real environment. On a perfect model (`sb26`, fidelity 1.0) planning reaches the target 1.00 of the time vs. 0.23 for random—the model is plannable. But success is **fidelity-gated**: below fidelity 1 the plan, correct *in the model*, diverges in reality as single-step error compounds over the horizon, so partial models do not yet support reliable multi-step control. *Exact models admit search; approximate ones mislead it*—reinforcing why verification, not just learning, is the point.

7 The solving wall: goal inference (E88–E90)

Solving (completing levels) is **unsolved**, and the value of this study is locating exactly why. (1) *Exploration cannot find a reward.* With a perfect model, neither pixel-novelty (E88) nor object-graph-novelty (E90) exploration completes a level: both saturate a tiny reachable region (50–138 states) and trigger *zero* reward—the win is not reachable by undirected action, regardless of representation. (2) *Goal hypotheses are sophisticated but unverifiable.* Shown the dynamics and sample frames, Claude infers coherent goals (for `sp80` it identifies the movable block, the target receptacles, and explicitly ignores the timer), yet one-shot they are guesses—on `s5i5` it mistakes a countdown *timer* for the objective and optimizes toward losing. (3) *Closing the loop is not enough.* Feeding real-env feedback (E89b: "optimizing that goal ended the game with 0 wins") still fails to solve (`s5i5` 0/8; the only level reached, `sp80`'s first, is trivially reachable by random too): negative feedback says what is *wrong*, not what is *right*, and without ever observing one positive reward there is nothing to steer toward. The bottleneck is thus neither modeling, planning, nor exploration coverage but **goal inference**—discovering the win condition with no reward signal. This is the wall frontier agents also hit (∼0 on ARC-AGI-3), here precisely located: we argue it is the central open problem for verified-world-model agents, and that bootstrapping a single positive reward (then inducing the win condition as a piecewise rule) is the key next step.

8 Solving a level (E93): directed beats random

Closing the loop from modeling to a genuine completion. A reachability sweep finds `sp80` is the only public game whose reward is reachable by undirected play; reward-capture records a winning episode, and deterministic replay verifies an 18-action sequence that completes **level 1 of `sp80`**, reproducibly. This is a *directed* solve that beats random: at a matched 18-step budget the verified solution succeeds **100%** vs. **9%** for random (300 trials). *Honest limits:* level 2 is unreachable even by a 600k-step biased search, and the solve is a verified action *sequence*, not a reverse-engineered win *rule* – the goal-inference wall (**§??** effectively) holds for the harder levels. Modeling is solved; intent is not.

Scaling to more games (E99): 6 solved. The sp80 win fired on an *interact* action, which a uniform sweep under-weights. An interact-biased, multi-seed reward search (capture → deterministic replay-verify) per game raises the count from 1 to **6/25 verified level-1 solves**: `ar25`, `cd82`, `ls20`, `sk48`, `sp80`, `tu93` (winning sequences 17–309 actions, all saved and replay-verified). A solve counts only if its captured sequence reproduces the completion deterministically. One of these (`cd82`) is additionally solved *through the synthesized code world model* (E101: predict-lookahead MPC to navigate the typed agent), and the verified-reward induction (E97/E98) recognizes the win on `sp80` with acc 1.0 from the agent's *own* synthesized objective.

What the remaining ∼19 expose: a trilogy of goal-discovery negatives. No reward is found on the rest even at 25k×4-seed interact-biased search: their wins are not *reachable*, they must be *understood*. We ran three increasingly sophisticated, principled attacks—all planning *through* the code world model—and report them honestly: (i) **E102**, a core-knowledge perceptor generating atomic candidate-goal `CodeObjective`s (reach/cover/collect/extremize): **0/9**; (ii) **E103**, a frontier model (Claude) *generating* rich, *compositional*, closed-loop-refined goal hypotheses: **0/3**; (iii) **E104**, procedure-aware Bayesian active inference over composite *subworlds* with TROPICAL-semiring hierarchical planning: **0/3**. None succeed *even with a fidelity*-1.0 *model*. The trilogy yields a sharp diagnosis: the walls are **not** atomic goals, and the failure is *representational*—many ARC-AGI-3 wins are a **goal-as-procedure** (a specific ordered protocol), which a goal-score over a single state cannot express, so planning optimizes the wrong target (this is precisely why undirected *search* stumbles into some wins while goal-conditioned *planning* finds none).

The one qualitatively-different lever: a visual perceptor (E106). Every method above fed the model text/structured state. In OpenWorld form we instead render each frame in the official palette and perceive it with a genuine `VisionPerceptor` (Claude-vision wrapped as a `BaseLLM`), feeding a `World` (the code world model as a `FunctionTransition`). This is the one representation that recovers *semantic, gestalt* goal hypotheses the structured methods cannot—e.g. for `ka59` it reads "two rooms joined by a corridor, a purple door/barrier, a green agent, a target square" and infers "cross the door to reach the target." Whether vision-guided planning completes a walled level is the live frontier test; the contribution is the OpenWorld-format pipeline (visual perception + verified world model + planning) and the honest characterization.

Honest framing. We claim a *search-vs-reasoning gap*: directed search reproducibly completes 6 level-1s where a frontier-agent metric sits near 1.5%, exposing that the benchmark's hardness for agents is *goal discovery* (specifically goal-as-procedure), not reachability—and we show a comprehensive, principled goal-discovery toolkit does *not* close it. This is **not** an agent state-of-the-art claim, which would require running under the leaderboard's live-agent protocol.

9 Reproducibility

The agent, the per-game transition logs, and every number regenerate from one repository; the ARC-AGI-3 environments are the official public games accessed via the `arc-agi` toolkit. Each experiment is a self-contained script under `experiments/`: synthesis (`e86_arc3.py`, `e86b_agentic.py`), planning/controllability (`e87_arc3_plan.py`), the solving pipeline—reward capture (`e93_capture_reward.py`), deterministic replay-verify (`e93b_replay_solve.py`), directed-vs-random (`e93d_directed_vs_random.py`), and the 6-game interact-biased sweep (`e99_solve_sweep.py`)—the world-model-driven solver (`e101_model_solver.py`), and the goal-discovery attack (`e102_-`

goal_search.py). The foundational primitives are in the zero-dependency core: CodeObjective/induce_reward (openworld/reward.py), ConsensusTransition (consensus.py), and make_typed_perceptor (perceive.py). **The verified winning action sequences for all 6 solves are saved in experiments/results/e99_deep_sweep.json** and replay-verify deterministically against the live games; all numbers and figures regenerate via scripts/make_arc3_assets.py. A one-command check, bench/verify_arc3_solves.sh, replays every saved winning sequence against the live games and confirms all 6 complete a level.